Plan de Explicación del Proyecto de Calculadora Móvil

I. Visión General del Proyecto

Este proyecto implementa una aplicación de calculadora móvil para Android, desarrollada con Kotlin y Jetpack Compose. La aplicación permite a los usuarios realizar operaciones aritméticas básicas y ver un historial de sus cálculos.

- **Propósito**: Proporcionar una calculadora funcional y fácil de usar con capacidad de historial de operaciones.
- **Tecnología Principal**: Android (Kotlin), Jetpack Compose para la interfaz de usuario.
- **Motor de Evaluación**: Utiliza el motor JavaScript Rhino para evaluar expresiones matemáticas, lo que permite una evaluación flexible y robusta de cadenas de texto.

II. Decisiones de Diseño y Patrones Utilizados

El proyecto sigue principios de diseño moderno de Android, buscando modularidad, reactividad y una UI declarativa.

1. **Jetpack Compose para la UI**
   - **Decisión**: Se optó por Jetpack Compose debido a su enfoque declarativo, que simplifica la construcción de interfaces de usuario complejas y reactivas en Android. Facilita la creación de componentes reutilizables y mejora la legibilidad del código UI.
   - **Patrón**: UI Declarativa. La interfaz se describe en función del estado de la aplicación, y Compose se encarga de actualizarla de manera eficiente cuando el estado cambia.

2. **Patrón MVVM (Model-View-ViewModel)**
   - **Decisión**: Implementar MVVM para separar claramente la lógica de negocio de la interfaz de usuario. Esto mejora la mantenibilidad, la capacidad de prueba y la escalabilidad del código.
   - **Patrón**:
     - **View (Vistas Composable)**: `Calculadora.kt` y `Historial.kt` son los componentes UI que observan los datos del ViewModel y reaccionan a las interacciones del usuario.
     - **ViewModel (`ModeloVistaCalculadora.kt`)**: Actúa como un intermediario entre el Modelo y la Vista. Contiene la lógica de la UI y expone datos observables (usando `LiveData`) que la Vista puede consumir.
     - **Model (`EvaluadorCalculadora.kt`)**: Contiene la lógica de negocio pura, como la evaluación de expresiones matemáticas.

3. **Manejo de Estado con `LiveData`**
   - **Decisión**: Se utiliza `LiveData` de Android Architecture Components para la gestión y observación del estado de la UI. `LiveData` es consciente del ciclo de vida de los componentes de Android, lo que ayuda a prevenir fugas de memoria y asegura que las actualizaciones de la UI solo ocurran cuando el componente esté activo.
   - **Patrón**: Observador. Las vistas "observan" los cambios en el `LiveData` expuesto por el ViewModel y se actualizan automáticamente.

4. **Navegación con Jetpack Compose Navigation**
   - **Decisión**: Se emplea la biblioteca de navegación de Jetpack Compose para manejar la transición entre la pantalla de la calculadora y la pantalla del historial.
   - **Patrón**: Navegación Basada en Componentes (Composable). Se definen rutas (`composable`) y se utiliza un `NavController` para navegar entre ellas,

manteniendo el estado de la navegación.

5. **Motor JavaScript Rhino para Evaluación de Expresiones**
   - **Decisión**: Integrar Rhino, un motor JavaScript, para la evaluación de expresiones matemáticas. Esto permite una implementación flexible de la lógica de evaluación, sin tener que escribir un *parser* y *evaluador* de expresiones desde cero en Kotlin.
   - **Implicaciones**: Simplifica la lógica de evaluación y permite manejar expresiones complejas con mayor facilidad, aunque introduce una dependencia externa y el costo de iniciar un contexto JavaScript.

6. **Theming de Material Design 3**
   - **Decisión**: Adoptar Material Design 3 para un aspecto moderno y consistente de la aplicación, utilizando los componentes y directrices más recientes de Android.
   - **Patrón**: Sistema de Diseño. Se definen colores, tipografías y formas en archivos de tema (`Color.kt`, `Theme.kt`, `Type.kt`, `themes.xml`) para aplicar un estilo uniforme.

III. Explicación de Cada Archivo Creado

A continuación, se detalla el funcionamiento de cada archivo clave en el proyecto.

1. `app/src/main/java/com/craos6518/calulcadoraprograacionmovil/MainActivity.kt`

- **Propósito**: Es el punto de entrada de la aplicación Android. Configura el tema de la aplicación, la navegación y el ViewModel principal.
- **Contenido / Función**:
  - Extiende `ComponentActivity`, la actividad base para aplicaciones Compose.
  - En `onCreate`, inicializa `ModeloVistaCalculadora` usando `ViewModelProvider`.
  - Configura `Jetpack Compose Navigation` para alternar entre la pantalla de la `Calculadora` y el `Historial`.
  - Aplica el tema `CalulcadoraPrograacionMovilTheme` a toda la aplicación.
- **Ejemplo de Código**:
```kotlin
package com.craos6518.calulcadoraprograacionmovil

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.ui.Modifier
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.lifecycle.ViewModelProvider
import
com.craos6518.calulcadoraprograacionmovil.ui.theme.CalulcadoraPrograacionMovilTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val modeloVistaCalculadora = ViewModelProvider(this)
[ModeloVistaCalculadora::class.java]
        enableEdgeToEdge()
```

```
        setContent {
            CalulcadoraProgramacionMovilTheme {
                val navController = rememberNavController()
                NavHost(navController = navController, startDestination =
"calculator") {
                    composable("calculator") {
                        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding
->
                            Calculadora(
                                modifier = Modifier.padding(innerPadding),
                                viewModel = modeloVistaCalculadora,
                                onOpenHistory = { navController.navigate("history")
}
                            )
                        }
                    }
                    composable("history") {
                        Historial(
                            viewModel = modeloVistaCalculadora,
                            onBack = { navController.popBackStack() }
                        )
                    }
                }
            }
        }
    }
}
```

2. `app/src/main/java/com/craos6518/calulcadoraprogramacionmovil/Calculadora.kt`

- **Propósito**: Define la interfaz de usuario (UI) principal de la calculadora.
- **Contenido / Función**:
  - Es un Composable que toma un `ModeloVistaCalculadora` como argumento para interactuar con la lógica.
  - Muestra la ecuación actual y el resultado (o mensaje de error) en dos campos de texto.
  - Renderiza una cuadrícula de botones para los números y operadores.
  - Incluye un botón para navegar a la pantalla del historial.
  - `CalculadoraBoton`: Un Composable reutilizable para cada botón de la calculadora, con lógica para asignar colores según el tipo de botón.
- **Ejemplo de Código**:
```kotlin
package com.craos6518.calulcadoraprogramacionmovil

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.aspectRatio
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
import androidx.compose.foundation.lazy.grid.items
import androidx.compose.foundation.shape.CircleShape
```

```
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.livedata.observeAsState
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.craos6518.calulcadoraprograacionmovil.ui.theme.CalculatorControlColor
import com.craos6518.calulcadoraprograacionmovil.ui.theme.CalculatorNumberColor
import com.craos6518.calulcadoraprograacionmovil.ui.theme.CalculatorOperatorColor
import
com.craos6518.calulcadoraprograacionmovil.ui.theme.CalulcadoraPrograacionMovilTheme

val botonesLista = listOf(
    "C", "AC", "%", "÷",
    "7", "8", "9", "×",
    "4", "5", "6", "-",
    "1", "2", "3", "+",
    "+/-", "0", ",", "="
)

@Composable
fun Calculadora(
    modifier: Modifier = Modifier,
    viewModel: ModeloVistaCalculadora,
    onOpenHistory: () -> Unit = {}
) {
    val ecuacionText by viewModel.ecuacionText.observeAsState("")
    val resultadosText by viewModel.resultadosText.observeAsState("0")
    val hayError by viewModel.isError.observeAsState(false)

    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.End
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.End,
            verticalAlignment = Alignment.CenterVertically
        ) {
            TextButton(onClick = onOpenHistory) {
                Text(text = stringResource(R.string.calculator_history))
            }
        }

        Spacer(modifier = Modifier.weight(1f))
```

```
        Text(
            text = ecuacionText,
            style = TextStyle(
                fontSize = 32.sp,
                textAlign = TextAlign.End,
            ),
            maxLines = 3,
            overflow = TextOverflow.Ellipsis,
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(8.dp))

        Text(
            text = if (hayError) {
                stringResource(R.string.error_matematico)
            } else {
                resultadosText
            },
            style = TextStyle(
                fontSize = 56.sp,
                fontWeight = FontWeight.Bold,
                textAlign = TextAlign.End,
            ),
            maxLines = 2,
            overflow = TextOverflow.Ellipsis,
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(16.dp))

        LazyVerticalGrid(
            columns = GridCells.Fixed(4),
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(8.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(botonesLista) { btn ->
                CalculadoraBoton(
                    btn = btn,
                    clickBoton = { viewModel.clickBoton(btn) }
                )
            }
        }
    }
}

@Composable
fun CalculadoraBoton(btn: String, clickBoton: () -> Unit) {
    Box(
        modifier = Modifier
            .aspectRatio(1f)
            .padding(2.dp)
    ) {
        FloatingActionButton(
            onClick = clickBoton,
            modifier = Modifier.fillMaxSize(),
            shape = CircleShape,
```

```
            containerColor = obtenerColor(btn),
            contentColor = Color.White
        ) {
            Text(
                text = btn,
                fontSize = 22.sp,
                fontWeight = FontWeight.Bold
            )
        }
    }
}

fun obtenerColor(btn: String): Color {
    return when (btn) {
        "C", "AC", "%" -> CalculatorControlColor
        "÷", "×", "-", "+", "=" -> CalculatorOperatorColor
        else -> CalculatorNumberColor
    }
}
```

3. `app/src/main/java/com/craos6518/calulcadoraprograacionmovil/Historial.kt`

- **Propósito**: Define la interfaz de usuario para mostrar el historial de operaciones de la calculadora.
- **Contenido / Función**:
  - Es un Composable que observa la lista de operaciones del `ModeloVistaCalculadora`.
  - Muestra las operaciones en un `LazyColumn` (lista eficiente para grandes cantidades de datos).
  - Proporciona un botón para volver a la pantalla de la calculadora (`onBack`).
  - Ofrece un botón para limpiar todo el historial.
  - Muestra un mensaje si el historial está vacío.
- **Ejemplo de Código**:
```kotlin
package com.craos6518.calulcadoraprograacionmovil

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.livedata.observeAsState
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
```

```
import
com.craos6518.calulcadoraprograacionmovil.ui.theme.CalulcadoraPrograacionMovilTheme

@Composable
fun Historial(
    viewModel: ModeloVistaCalculadora,
    onBack: () -> Unit
) {
    val operaciones by viewModel.historial.observeAsState(emptyList())

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            TextButton(onClick = onBack) {
                Text(stringResource(R.string.history_title))
            }
            Text("Historial", style = MaterialTheme.typography.headlineSmall)
            TextButton(
                onClick = viewModel::limpiarHistorial,
                enabled = operaciones.isNotEmpty()
            ) {
                Text(stringResource(R.string.history_clear))
            }
        }

        if (operaciones.isEmpty()) {
            Column(
                modifier = Modifier.fillMaxSize(),
                verticalArrangement = Arrangement.Center,
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(stringResource(R.string.history_empty))
                Button(onClick = onBack, modifier = Modifier.padding(top = 12.dp))
{
                    Text(stringResource(R.string.history_return))
                }
            }
        } else {
            LazyColumn(
                modifier = Modifier.fillMaxSize(),
                verticalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                items(operaciones) { operacion ->
                    Card(modifier = Modifier.fillMaxWidth()) {
                        Text(
                            text = operacion,
                            modifier = Modifier.padding(16.dp),
                            style = MaterialTheme.typography.titleMedium
                        )
                    }
                }
            }
```

```
        }
    }
}

@Preview(showBackground = true, name = "Historial Vacio")
@Composable
fun HistorialVacioPreview() {
    val viewModel = remember { ModeloVistaCalculadora() }
    CalulcadoraPrograacionMovilTheme {
        Historial(
            viewModel = viewModel,
            onBack = {}
        )
    }
}

@Preview(showBackground = true, name = "Historial Con Datos")
@Composable
fun HistorialConDatosPreview() {
    val viewModel = remember {
        ModeloVistaCalculadora().apply {
            clickBoton("1")
            clickBoton("+")
            clickBoton("2")
            clickBoton("=")
            clickBoton("5")
            clickBoton("×")
            clickBoton("3")
            clickBoton("=")
        }
    }
    CalulcadoraPrograacionMovilTheme {
        Historial(
            viewModel = viewModel,
            onBack = {}
        )
    }
}
```

4. `app/src/main/java/com/craos6518/calulcadoraprograacionmovil/ModeloVistaCalculadora.kt`

- **Propósito**: Es el ViewModel principal de la aplicación, que contiene la lógica de negocio relacionada con la calculadora y gestiona el estado de la UI.
- **Contenido / Función**:
  - Extiende `ViewModel`, lo que le permite sobrevivir a los cambios de configuración de Android (como la rotación de pantalla).
  - Expone `LiveData` para `ecuacionText` (lo que se muestra en la pantalla como entrada), `resultadosText` (el resultado actual o parcial), `isError` (bandera para indicar un error) y `historial` (la lista de operaciones).
  - Contiene una instancia de `EvaluadorCalculadora` para realizar los cálculos.
  - `clickBoton(boton: String)`: Es el método central que maneja todos los clics de los botones de la calculadora, actualizando el estado de la ecuación y el resultado.
  - Implementa lógica para limpiar la calculadora, añadir decimales, cambiar el signo del último número y actualizar el resultado parcial.
- **Ejemplo de Código**:

```kotlin
package com.craos6518.calulcadoraprograacionmovil

import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class ModeloVistaCalculadora : ViewModel() {

    private val _ecuacionText = MutableLiveData("")
    val ecuacionText: LiveData<String> = _ecuacionText

    private val _resultadoText = MutableLiveData("0")
    val resultadosText: LiveData<String> = _resultadoText

    private val _isError = MutableLiveData(false)
    val isError: LiveData<Boolean> = _isError

    private val _historial = MutableLiveData<List<String>>(emptyList())
    val historial: LiveData<List<String>> = _historial

    private val evaluador = EvaluadorCalculadora()
    private var resultadoConfirmado = false

    fun limpiarHistorial() {
        _historial.value = emptyList()
    }

    fun clickBoton(boton: String) {
        if (_isError.value == true) {
            _ecuacionText.value = ""
            _resultadoText.value = "0"
            _isError.value = false
            resultadoConfirmado = false
        }
        var expresion = _ecuacionText.value.orEmpty()

        when (boton) {
            "AC" -> {
                limpiarCalculadora()
                return
            }
            "C" -> {
                if (expresion.isNotEmpty()) {
                    expresion = expresion.dropLast(1)
                    _ecuacionText.value = expresion
                    resultadoConfirmado = false
                    actualizarResultadoParcial(expresion)
                }
                return
            }
            "=" -> {
                if (resultadoConfirmado || expresion.isBlank()) return

                when (val resultado = evaluador.evaluar(expresion)) {
                    is ResultadoEvaluacion.Exito -> {
                        val registro = "$expresion = ${resultado.valor}"
                        val historialActual = _historial.value.orEmpty()
```

```
                    if (historialActual.firstOrNull() != registro) {
                        _historial.value = listOf(registro) + historialActual
                    }

                    _resultadoText.value = resultado.valor
                    _ecuacionText.value = resultado.valor
                    _isError.value = false
                    resultadoConfirmado = true
                }

                ResultadoEvaluacion.Error -> {
                    _isError.value = true
                }
            }
            return
        }
        "+/-" -> {
            expresion = cambiarSignoDelUltimo(expresion)
            _ecuacionText.value = expresion
            resultadoConfirmado = false
            actualizarResultadoParcial(expresion)
            return
        }
        "," -> {
            expresion = agregarDecimal(expresion)
        }
        "%" -> {
            if (expresion.endsWith("%")) return
            expresion += "%"
        }
        "+", "-", "×", "÷" -> {
            if (expresion.isEmpty()) {
                if (boton != "-") return
            } else if (expresion.last() in "+-×÷") {
                expresion = expresion.dropLast(1)
            } else if (expresion.last() == ',') {
                return
            }
            expresion += boton
        }
        in "0".."9" -> {
            if (resultadoConfirmado) {
                expresion = ""
                _resultadoText.value = "0"
                resultadoConfirmado = false
            }
            expresion += boton
        }
        else -> return
    }
    _ecuacionText.value = expresion
    actualizarResultadoParcial(expresion)
}

private fun limpiarCalculadora() {
    _ecuacionText.value = ""
    _resultadoText.value = "0"
    _isError.value = false
    resultadoConfirmado = false
```

```
    }

    private fun agregarDecimal(expresion: String): String {
        val indiceOperador = expresion.indexOfLast { it in "+-×÷" }
        val ultimoNumero = expresion.substring(indiceOperador + 1)

        if (ultimoNumero.contains(",") || ultimoNumero.contains(".")) {
            return expresion
        }

        return if (expresion.isEmpty() || expresion.last() in "+-×÷") {
            "${expresion}0,"
        } else {
            "$expresion,"
        }
    }

    private fun cambiarSignoDelUltimo(expresion: String): String {
        if (expresion.isEmpty() || expresion.last() in "+-×÷") {
            return "$expresion-"
        }
        val ultimoNumero = Regex("""-?\d+(?:[,.]\d*)?$""").find(expresion)
            ?: return expresion

        val numero = ultimoNumero.value
        val reemplazo = if (numero.startsWith("-")) {
            numero.drop(1)
        } else {
            "(-$numero)"
        }
        return expresion.replaceRange(ultimoNumero.range, reemplazo)
    }

    private fun actualizarResultadoParcial(expresion: String) {
        if (expresion.isBlank() || expresion.last() in "+-×÷" || expresion == "-")
{
            _isError.value = false
            return
        }
        when (val resultado = evaluador.evaluar(expresion)) {
            is ResultadoEvaluacion.Exito -> {
                _resultadoText.value = resultado.valor
                _isError.value = false
            }
            ResultadoEvaluacion.Error -> {
                _isError.value = true
            }
        }
    }
}
```

5. `app/src/main/java/com/craos6518/calulcadoraprograacionmovil/EvaluadorCalculadora.kt`

- **Propósito**: Contiene la lógica para evaluar expresiones matemáticas.
- **Contenido / Función**:
  - Define una `sealed class` `ResultadoEvaluacion` para representar el éxito (con

un valor) o un error en la evaluación.
  - La clase `EvaluadorCalculadora` tiene el método `evaluar(expresion: String)`.
  - Este método utiliza el motor JavaScript Rhino para parsear y calcular el resultado de la expresión dada.
  - Realiza una pre-validación de caracteres y reemplaza operadores visuales (`×`, `÷`, `,`) por sus equivalentes de JavaScript (`*`, `/`, `.`) para que Rhino pueda interpretarlos correctamente.
  - Maneja posibles errores durante la evaluación, devolviendo `ResultadoEvaluacion.Error`.
- **Ejemplo de Código**:
```kotlin
package com.craos6518.calulcadoraprograacionmovil

import org.mozilla.javascript.Context

sealed class ResultadoEvaluacion {
    data class Exito(val valor: String) : ResultadoEvaluacion()
    data object Error : ResultadoEvaluacion()
}

class EvaluadorCalculadora {
    fun evaluar(expresion: String): ResultadoEvaluacion {
        if (expresion.isBlank()) {
            return ResultadoEvaluacion.Error
        }

        // Validar caracteres permitidos
        val caracteresPermitidos = Regex("""[0-9+\-×÷*/%().,\s]+""")
        if (!caracteresPermitidos.matches(expresion)) {
            return ResultadoEvaluacion.Error
        }

        // Reemplazar operadores visuales por operadores reconocidos por JavaScript
        val expresionParaRhino = expresion
            .replace("×", "*")
            .replace("÷", "/")
            .replace(",", ".")

        val contexto = Context.enter()
        return try {
            contexto.optimizationLevel = -1
            val scope = contexto.initStandardObjects()
            val objeto = contexto.evaluateString(scope, expresionParaRhino,
"Calculadora", 1, null)
            val numero = objeto.toString().toDoubleOrNull()

            if ((numero != null) && numero.isFinite()) {
                ResultadoEvaluacion.Exito(numero.toString())
            } else {
                ResultadoEvaluacion.Error
            }
        } catch (_: Exception) {
            ResultadoEvaluacion.Error
        } finally {
            Context.exit() // Liberar recursos de Rhino
        }
    }
}
```

6. `app/src/main/java/com/craos6518/calulcadoraprograacionmovil/ui/theme/Color.kt`

- **Propósito**: Define los colores utilizados en la aplicación, siguiendo la paleta de Material Design.
- **Contenido / Función**: Contiene objetos `Color` de Jetpack Compose con valores hexadecimales, usados para el theming. Incluye colores base de Material 3 (Purple, Pink, Grey) y colores específicos para la calculadora (control, operadores, números).
- **Ejemplo de Código**:
```kotlin
package com.craos6518.calulcadoraprograacionmovil.ui.theme

import androidx.compose.ui.graphics.Color

val Purple80 = Color(0xFFD0BCFF)
val PurpleGrey80 = Color(0xFFCCC2DC)
val Pink80 = Color(0xFFEFB8C8)

val Purple40 = Color(0xFF6650a4)
val PurpleGrey40 = Color(0xFF625b71)
val Pink40 = Color(0xFF7D5260)

val CalculatorControlColor = Color(0xFF5D5D5D)
val CalculatorOperatorColor = Color(0xFFFF9100)
val CalculatorNumberColor = Color(0xFF343434)
```

7. `app/src/main/java/com/craos6518/calulcadoraprograacionmovil/ui/theme/Theme.kt`

- **Propósito**: Define el tema de la aplicación, configurando el esquema de colores y tipografía de Material Design 3.
- **Contenido / Función**:
  - Define `DarkColorScheme` y `LightColorScheme` con colores primarios, secundarios y terciarios.
  - El Composable `CalulcadoraPrograacionMovilTheme` aplica el esquema de color adecuado según si el dispositivo está en modo oscuro o claro, y si soporta colores dinámicos (Material You en Android 12+).
  - Integra la tipografía definida en `Type.kt`.
- **Ejemplo de Código**:
```kotlin
package com.craos6518.calulcadoraprograacionmovil.ui.theme

import android.app.Activity
import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

private val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    secondary = PurpleGrey80,
```

```
    tertiary = Pink80
)

private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    secondary = PurpleGrey40,
    tertiary = Pink40
)

@Composable
fun CalulcadoraPrograacionMovilTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else
dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

8. `app/src/main/java/com/craos6518/calulcadoraprograacionmovil/ui/theme/Type.kt`

- **Propósito**: Define la tipografía utilizada en la aplicación.
- **Contenido / Función**: Configura un objeto `Typography` de Material 3, estableciendo el estilo `bodyLarge` como base con la fuente por defecto del sistema. Los demás estilos están comentados para ser personalizados si es necesario.
- **Ejemplo de Código**:
```kotlin
package com.craos6518.calulcadoraprograacionmovil.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

val Typography = Typography(
    bodyLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    )
```

)
```

### 9. `app/src/main/res/values/strings.xml`

- **Propósito**: Archivo de recursos de cadenas de texto para la aplicación.
- **Contenido / Función**: Define todas las cadenas de texto visibles en la UI, permitiendo su internacionalización (traducción) y centralizando el texto. Incluye nombres de la app, títulos de pantalla, etiquetas de botones y mensajes de error.
- **Ejemplo de Código**:
```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Calulcadora Progra,acion Movil</string>
    <string name="history_title">Historial</string>
    <string name="history_clear">Borrar</string>
    <string name="history_empty">Todavía no hay operaciones</string>
    <string name="history_return">Volver a calcular</string>
    <string name="calculator_history">Historial</string>
    <string name="error_matematico">ERROR Matemático</string>
</resources>
```

10. `app/src/main/res/values/colors.xml`

- **Propósito**: Archivo de recursos de colores para la aplicación.
- **Contenido / Función**: Define colores básicos usados en el tema XML de la aplicación (principalmente los colores por defecto del template de Android Studio).
- **Ejemplo de Código**:
```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

11. `app/src/main/res/values/themes.xml`

- **Propósito**: Define el tema XML base de la aplicación (para el sistema Android, no para Compose).
- **Contenido / Función**: Hereda de `Theme.Material.Light.NoActionBar`, lo que establece un tema Material Design por defecto sin barra de acción para la actividad principal.
- **Ejemplo de Código**:
```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="Theme.CalulcadoraPrograacionMovil"
parent="android:Theme.Material.Light.NoActionBar" />
</resources>
```

IV. Flujo de Ejecución y Funcionamiento
```

1. **Inicio de la App**: `MainActivity.kt` se inicia. Crea una instancia de `ModeloVistaCalculadora` y configura la navegación de Jetpack Compose.
2. **Pantalla de Calculadora (`Calculadora.kt`)**:
   - La UI observa `ecuacionText`, `resultadosText` y `isError` del `ModeloVistaCalculadora`.
   - Cuando el usuario presiona un botón, se llama a `viewModel.clickBoton(btn)`.
   - El `ModeloVistaCalculadora` actualiza su estado interno (`_ecuacionText`, `_resultadoText`, etc.) basado en el botón presionado.
   - `LiveData` notifica a la UI de los cambios, y Jetpack Compose re-renderiza automáticamente la interfaz para reflejar el nuevo estado.
   - Para operaciones matemáticas, `ModeloVistaCalculadora` utiliza `EvaluadorCalculadora` (que a su vez usa Rhino) para calcular el resultado.
3. **Pantalla de Historial (`Historial.kt`)**:
   - Al presionar el botón "Historial" en la calculadora, se navega a esta pantalla.
   - Observa `historial` del `ModeloVistaCalculadora` y muestra las operaciones en una lista.
   - Permite borrar el historial o volver a la calculadora.

V. Consideraciones Adicionales y Posibles Mejoras

- **Manejo de Errores**: El `EvaluadorCalculadora` captura excepciones de Rhino, pero podría proporcionar mensajes de error más específicos al usuario (ej. división por cero, sintaxis inválida).
- **Persistencia del Historial**: Actualmente, el historial se almacena en memoria (dentro del ViewModel) y se pierde al cerrar la aplicación. Para persistencia, se podría usar `Room` (base de datos SQLite) o `DataStore`.
- **Testing**: Se incluyen archivos de test unitario (`EvaluadorCalculadoraTest.kt`, `ModeloVistaCalculadoraTest.kt`) y de instrumentación (`CalculadoraUiTest.kt`, `ExampleInstrumentedTest.kt`), lo que indica un enfoque en la calidad del código.
- **Accesibilidad**: Se podría mejorar la accesibilidad añadiendo `contentDescription` a los botones y asegurando un tamaño de texto adecuado para lectores de pantalla.